

Arquivo: file_service.py

```
"""
DESCRIÇÃO GERAL:
Esta camada representa o serviço do coordenador
Ela recebe a requisição da CLI, decide qual nó deve atender a operação e encaminha a mensagem para o destino
"""

from dfs.cluster.node_client import NodeClient
from dfs.cluster.node_registry import NodeRegistry
from dfs.cluster.sharding import ShardingManager
from dfs.application.metadata_service import MetadataService
from dfs.protocol import parse_request, make_request, make_response
from dfs.config import CHUNK_SIZE

class FileService:
    """
    Serviço central do coordenador.

    Responsabilidades:
    - receber requisições da CLI
    - dividir arquivos em chunks no PUT
    - distribuir chunks entre storage nodes
    - registrar metadados
    - reconstruir arquivos no GET
    - remover chunks no DELETE
    - listar arquivos pelo índice
    """

    def __init__(
        self,
        registry: NodeRegistry | None = None,
        sharding: ShardingManager | None = None,
        metadata: MetadataService | None = None,
        timeout: float = 5.0,
    ):
        # Cadastro dos nós conhecidos
        self.registry = registry or NodeRegistry()

        # Mapeamento de shards para nós
        self.sharding = sharding or ShardingManager(self.registry)

        # Índice de arquivos e seus chunks
        self.metadata = metadata or MetadataService()

        # Timeout padrão para comunicação entre nós
        self.timeout = timeout

    # Enviar uma requisição para o nó responsável
    def _send_to_node(self, node, request_op: str, path: str, data: bytes = b"", shard_id: int = 0):
        """
        Encaminha a operação para o nó responsável
        """
```

```

"""
# Calcula o shard associado ao caminho
shard_id = self.sharding.shard_id_for_path(path)

# Cria cliente interno para falar com o nó.
client = NodeClient(node.host, node.port, timeout=self.timeout)

# Monta a requisição Protobuf que será enviada ao nó
raw_request = make_request(
    op=request_op,
    path=path, # caminho físico do chunk dentro do storage node
    data=data, # bytes (apenas para PUT, vazio para GET e DELETE)
    node_id=node.node_id,
    shard_id=shard_id,
)

# Envia a requisição já serializada
return client.send_raw(raw_request)

# Método auxiliar para dividir um arquivo em chunks
def _split_into_chunks(self, data: bytes) -> list[bytes]:

    # Cria uma lista vazia para armazenar os pedaços do arquivo
    chunks = []

    # Percorre os bytes do arquivo de CHUNK_SIZE em CHUNK_SIZE
    # Exemplo: se CHUNK_SIZE = 64 KB, pega blocos de 64 KB
    for start in range(0, len(data), CHUNK_SIZE):

        # Corta um pedaço dos bytes originais
        # Começa em "start" e vai até "start + CHUNK_SIZE"
        chunks.append(data[start:start + CHUNK_SIZE])

    # Caso o arquivo esteja vazio, ainda criamos um chunk vazio
    # Permite representar arquivos vazios no DFS
    if not chunks:
        chunks.append(b"")

    # Retorna a lista de chunks.
    return chunks

# Método que trata a operação PUT
def _put(self, request):

    # Verifica se o cliente informou um caminho lógico
    # Exemplo de caminho lógico: docs/aula.pdf
    if not request.path:

        # Se não houver caminho, retorna erro para a CLI, sem tentar falar com os nós
        return make_response(
            False,

```

```

        "Caminho lógico vazio",
        node_id="coordinator",
        shard_id=-1,
    )

# Divide o arquivo recebido em pedaços físicos
chunks = self._split_into_chunks(request.data)

# Lista que guardará os metadados de cada chunk salvo
# Cada item terá: chunk_id, node_id, shard_id, chunk_path e size
chunk_metadata = []

try:
    # enumerate fornece:
    # chunk_id = posição do chunk
    # chunk_data = bytes daquele pedaço
    for chunk_id, chunk_data in enumerate(chunks):

        # Calcula o shard responsável por este chunk
        shard_id = self.sharding.shard_for_chunk(request.path, chunk_id)

        # Descobre qual storage node corresponde ao chunk
        node = self.sharding.node_for_chunk(request.path, chunk_id)

        # Gera o caminho físico onde o chunk será salvo no nó
        # Exemplo: .chunks/docs_aula_pdf/chunk_000000
        chunk_path = self.sharding.chunk_storage_path(request.path, chunk_id)

        # Envia o chunk para o storage node escolhido
        response = self._send_to_node(
            node=node,
            op="PUT",
            path=chunk_path,
            data=chunk_data,
            shard_id=shard_id,
        )

        # Verifica se o storage node conseguiu salvar o chunk
        if not response.ok:

            # Se falhou, retorna erro para a CLI
            return make_response(
                False,
                f"Falha ao salvar chunk {chunk_id}: {response.message}",
                node_id=node.node_id,
                shard_id=shard_id,
            )

        # Se salvou corretamente, registra as informações do chunk
        chunk_metadata.append({

            # Número sequencial do chunk

```

```

        # Necessário para reconstruir o arquivo na ordem correta
        "chunk_id": chunk_id,

        # Caminho físico do chunk no storage node
        "chunk_path": chunk_path,

        # Nó onde o chunk foi salvo
        "node_id": node.node_id,

        # Shard associado ao chunk
        "shard_id": shard_id,

        # Tamanho do chunk em bytes
        "size": len(chunk_data),
    })

    # Depois que todos os chunks foram salvos, registra o arquivo no índice
    # Esse é o ponto principal da indexação por metadados
    self.metadata.put_file(
        path=request.path,
        size=len(request.data),
        chunks=chunk_metadata,
    )

    # Retorna sucesso ao cliente
    return make_response(
        True,
        f"Arquivo salvo com {len(chunks)} chunk(s) distribuído(s)",
        node_id="coordinator",
        shard_id=-1,
    )

    # Se qualquer erro inesperado ocorrer durante o PUT, cai aqui
    except Exception as exc:

        # Retorna erro controlado.
        return make_response(
            False,
            f"Erro no PUT distribuído: {exc}",
            node_id="coordinator",
            shard_id=-1,
        )

# Método que trata a operação GET
def _get(self, request):

    # Verifica se o caminho lógico foi informado
    if not request.path:

        # Se não foi, retorna erro
        return make_response(

```

```

        False,
        "Caminho lógico vazio",
        node_id="coordinator",
        shard_id=-1,
    )

# Consulta o índice de metadados para descobrir onde estão os chunks
metadata = self.metadata.get_file(request.path)

# Se o arquivo não está no índice, o coordenador não sabe onde buscá-lo
if metadata is None:

    # Retorna erro de arquivo inexistente
    return make_response(
        False,
        "Arquivo não encontrado no índice de metadados",
        node_id="coordinator",
        shard_id=-1,
    )

# Ordena os chunks pelo chunk_id
# Isso é essencial para reconstruir o arquivo original corretamente
chunks = sorted(metadata["chunks"], key=lambda item: item["chunk_id"])

# Lista que armazenará os bytes de cada pedaço recuperado
file_parts = []

try:
    for chunk in chunks:
        # Descobre o nó onde aquele chunk foi salvo
        node = self.registry.get(chunk["node_id"])

        # Pede ao nó para ler o chunk físico
        response = self._send_to_node(
            node=node,
            op="GET",
            path=chunk["chunk_path"],
            shard_id=chunk["shard_id"],
        )

        # Se o nó não conseguiu devolver o chunk, retorna erro
        if not response.ok:

            # Informa exatamente qual chunk falhou
            return make_response(
                False,
                f"Falha ao recuperar chunk {chunk['chunk_id']}: {response.message}",
                node_id=node.node_id,
                shard_id=chunk["shard_id"],
            )

        # Adiciona os bytes do chunk à lista de partes

```

```

        file_parts.append(response.data)

# Junta todos os chunks na ordem correta
full_data = b"".join(file_parts)

# Retorna o arquivo reconstruído ao cliente
return make_response(
    True,
    "Arquivo reconstruído com sucesso",
    data=full_data,
    node_id="coordinator",
    shard_id=-1,
)

except Exception as exc:

    return make_response(
        False,
        f"Erro no GET distribuído: {exc}",
        node_id="coordinator",
        shard_id=-1,
    )

# Método que trata a operação DELETE
def _delete(self, request):

    # Verifica se o caminho lógico foi informado
    if not request.path:

        # Se não foi, retorna erro
        return make_response(
            False,
            "Caminho lógico vazio",
            node_id="coordinator",
            shard_id=-1,
        )

    # Busca os metadados do arquivo no índice
    metadata = self.metadata.get_file(request.path)

    # Se não existir no índice, não há como saber quais chunks apagar
    if metadata is None:

        return make_response(
            False,
            "Arquivo não encontrado no índice",
            node_id="coordinator",
            shard_id=-1,
        )

    # Lista para guardar erros parciais
    # Exemplo: chunk 0 apagou, mas chunk 1 falhou

```

```
errors = []
```

```
# Percorre cada chunk do arquivo
```

```
for chunk in metadata["chunks"]:
```

```
    # Usa try porque cada nó pode falhar independentemente
```

```
    try:
```

```
        # Recupera o nó onde o chunk está armazenado
```

```
        node = self.registry.get(chunk["node_id"])
```

```
        # Envia DELETE ao nó responsável pelo chunk
```

```
        response = self._send_to_node(
```

```
            node=node,
```

```
            op="DELETE",
```

```
            path=chunk["chunk_path"],
```

```
            shard_id=chunk["shard_id"],
```

```
        )
```

```
        # Se o nó respondeu erro, registra a falha
```

```
        if not response.ok:
```

```
            errors.append(f"chunk {chunk['chunk_id']}: {response.message}")
```

```
    # Se houve erro de conexão ou outro erro, registra a falha
```

```
    except Exception as exc:
```

```
        errors.append(f"chunk {chunk['chunk_id']}: {exc}")
```

```
# Se houve qualquer erro, não remove o metadado
```

```
# Evita o coordenador esquecer um arquivo que ainda tem pedaços sobrando
```

```
if errors:
```

```
    return make_response(
```

```
        False,
```

```
        "Falha parcial ao remover arquivo: " + "; ".join(errors),
```

```
        node_id="coordinator",
```

```
        shard_id=-1,
```

```
    )
```

```
# Se todos os chunks foram removidos com sucesso, remove também a entrada do arquivo no índice de
```

```
self.metadata.delete_file(request.path)
```

```
# Retorna sucesso
```

```
return make_response(
```

```
    True,
```

```
    "Arquivo removido e metadados atualizados",
```

```
    node_id="coordinator",
```

```
    shard_id=-1,
```

```
)
```

```
# Método que trata a operação LIST.
```

```
def _list(self):
```

```

# Lista os arquivos diretamente do índice de metadados
# O LIST mostra os arquivos lógicos conhecidos pelo DFS, e não os chunks físicos espalhados nos n
entries = self.metadata.list_files()

# Retorna a lista ao cliente
return make_response(
    True,
    "Listagem feita a partir do índice de metadados",
    entries=entries,
    node_id="coordinator",
    shard_id=-1,
)

```

O server.py chama este método sempre que recebe uma mensagem da CLI

```
def dispatch(self, raw_request: bytes) -> bytes:
```

```

# Converte os bytes recebidos pela rede em uma requisição Protobuf
request = parse_request(raw_request)

```

```

# Normaliza a operação
op = request.op.upper().strip()

```

```

try:
    if op == "PUT":
        return self._put(request)

    if op == "GET":
        return self._get(request)

    if op == "DELETE":
        return self._delete(request)

    if op == "LIST":
        return self._list()

```

```

# Se a operação não for reconhecida, retorna erro
return make_response(
    False,
    "Operação inválida",
    node_id="coordinator",
    shard_id=-1,
)

```

```

# Captura qualquer erro inesperado dentro do coordenador
except Exception as exc:

```

```

# Retorna uma resposta Protobuf de erro
# Isso evita que o servidor quebre sem responder ao cliente
return make_response(
    False,
    f"Erro inesperado no coordenador: {exc}",
    node_id="coordinator",
)

```



```
    shard_id=-1,  
)
```